

Task 4 – Pattern Justification

Keywords

- **Engine** = Workflow management system
- **Work item** = Individual piece of work moving through the workflow
 - Like a task needed to be complete (Claim verification, Phone battery assembly)
- **Workflow Definition** = The blueprint, template, or recipe of a process. It basically defines what steps exist, what order they run in, their rules, and who performs them
- **Workflow Instance** = One actual, running execution of that blueprint (e.g., Claim #1042 currently underway).

Factory Method

Problem in TIVIDY

- Different processes need different kinds of work items or incoming triggers (like a human task, an automatic step). If the engine created each type with **new ConcreteX()** everywhere, it would have to know every type and adding new types would mean editing the engine, thus violating the **OCP**

Participants

- InputCreator (**Creator**, it declares the factory method)
- ConcreteInputCreator (**ConcreteCreator**)
- Input (**Product**)
- ConcreteInput (**ConcreteProduct**)

Collaboration

- The engine essentially asks the InputCreator for an Input. The ConcreteCreator decides which type of ConcreteInput to make and returns it as an Input, so the engine only sees the general type (**Polymorphism**)

What varies

- The kind of work item or trigger that gets created (even though the factory method return an Input, the execution for each is not the same)

Why appropriate

- The engine has to stay generic. The factory method lets a new process bring its own types of input without any change to the engine code

Observer

Problem in TIVIDY

- When something happens to a piece of work (it is either completed, rejected or delivered), several other parts need to be informed, the next work item must become available, people must be told about it and the history must be updated. If the work item called each of the directly, it would be tied to all of them (it would have to keep track of all of them)

Participants

- Input (**Subject**)
- CompositeNotifier (**Observer**)
- ConcreteCompositeNotifier (**ConcreteObserver**)

Collaboration

- Notifiers register with an Input (Work item). When the Input changes, it informs every registered notifier and each one react in it own way

What varies

- Which parts are listening and what each one does

Why appropriate

- The spec says completing one activity may make another available, an external event may trigger new work, and several parts may need to react to the same event. This is one-to-many, which is what Observer is for.

Adapter

Problem in TIVIDY

- Workflows must talk to outside systems whose interfaces were not designed for TIVIDY (for example a payment service, an inventory system or a permit office). The engine should basically not depend on any one vendor's interface.

Participants

- Input (**Target**)
- ConcreteInput (**Adapter**)
- Adaptee (**Adaptee**)

Collaboration

- The engine uses the general Input interface. The adapting ConcreteInput then translates those calls into the outside system's own calls (the Adaptee) and translates the answers back

What varies

- The outside system being connected

Why appropriate

- Connecting a new system just mean writing a new adapter and that has no changes to the engine

Builder

Problem in TIVIDY

- A work item or workflow part has many settings, like names, rules, what it depends on, approval needs etc. Setting all these in one giant constructor is essentially error prone and would be hard to read and the steps differ between processes

Participants

- ItemManager (**Builder**)
- ConcreteItemManager (**ConcreteBuilder**)
- Director (**Director**)
- ConcreteInput (**Product**)

Collaboration

- The Director manages the construction steps in a fixed order. ConcreteItemManager carried out each step and give back a finished product

What varies

- The settings and the way in which a work item is assembled

Why appropriate

- A workflow definition is assembled in many steps and the same steps can produce different results for different processes

Note: The builder would take a factory which creates a type of Input (Work item), like HumanTask or AutomatedTask, and the builder configures it, like adding priorities, deadlines etc.

State

Problem in TIVIDY

- A work item changes states over time (created, available, assigned, in progress, completed, cancelled) and the actions that are valid depend on where its current state is. For example, a cancelled item must not be completable. Checking the current status field with large if/else chains breaks the spec's rules against that

Participants

- AbstractState (**State**)
- ConState (**ConcreteState**)
- Context

Collaboration

- The work item hands each action to its current state object. The state will either allow the action and moves the item to the next state or refuse it

What varies

- The lifecycle rules, meaning which state exist and which actions each state allows

Why appropriate

- Each rule lives in the state that owns it, so that means adding a state, like **escalated**, **does** not touch others

Composite

Problem in TIVIDY

- Work is not flat. A process has stages it can go through, which contains tasks, which may also contain sub-processes. The engine should be to start, complete or cancel a single task or a whole stage the same way without checking which it is

Participants

- Director (Component)
- Leaf (Leaf)
- AggregateCollaborators (Composite), plus the classes the group marked as composite leaves (CompositeNotifier, ItemManager, ConcreteAuthorizer, Collaborator).

Collaboration

- A group passes each request down to its children. Leaves do the real work

What varies

- The depth and shape of the work structure

Why appropriate

- This is exactly the "stages, work items and sub-processes" structure in the spec.

Iterator

Problem in TIVIDY

- The engine has to go through a group of parts one by one (for example to find the next available work item or to collect progress), without depending on how the group stores them.

Participants

- AggregateCollaborators (**Aggregate**)
- Iterator (**Iterator**).

Collaboration

- The aggregate creates an iterator. The engine asks it for the next element until it is finished.

What varies

- How the group is stored and in what order it is traversed

Why appropriate

- The specs asks for navigation of nested work “without requiring the rest of the system to know their internal storage structure”

Decorator

Problem in TIVIDY

- A work item may need extra behaviour (like priority handling, validation, notification) without becoming a different kind of item. Subclasses for every combination would multiply very fast

Participants

- Configuration (**Decorator**)
- ConcreteConfiguration (**ConcreteDecorator**)
- and the Component that is wrapped

Collaboration

- A decorator wraps a work item, does its extra step before or after passing the call to the item it wraps, and decorators can be stacked

What varies

- Which extra behaviour a given item has

Why appropriate

- The spec names this exactly. For example, the oldest waiting item in a queue can be wrapped with extra priority handling

Memento

Problem in TIVIDY

- The system must be able to undo or roll back when something goes wrong. Rolling back needs the earlier state but outside classes shouldn't be able to read or edit a workflows internals

Participants

- Director (**Originator**)
- Processor (**Memento**) *WorkflowSnapshot*
- Distributor (**Caretaker**) *SnapshotHistory*

Collaboration

- Before a change, the Originator creates a memento of its state. The caretaker stores it without looking inside. To undo, the caretaker hands the memento back and the Originator restores itself.

What varies

- What is saved and how many snapshots are kept

Why appropriate

- The spec asks for retained history and rollback where sensible, and this keeps the saved state hidden.

Mediator

Problem in TIVIDY

- The engine has many cooperating parts (notifiers, builders, collections of collaborators). If each called the others directly, every part would depend on every other, and a new part would mean changing many classes(not very efficient)

Participants

- Communication (**Mediator**)
- ConcreteCommunication (**ConcreteMediator**)
- Collaborators (**Colleague**) with ConcreteCompositeNotifier, ConcreteItemManager, AggregateCollaborators, ConcreteAuthorizer and Collaborator as colleagues.

Collaboration

- Parts send messages only to the mediator, which decides who needs to hear about them. For example, when an approval step finishes, it tells the mediator, which tells the notifier and the item manager

What varies

- Who talks to whom

Why appropriate

- The spec asks that concerns be separated from the core workflow model so that new rules, interested parties or integrations can be introduced without redesigning the entire engine

Chain of Responsibility

Problem in TIVIDY

- Approval may involve one person, several levels, or escalation when nobody responds. Different processes have different levels and limits, and these should not be hard-coded into the work item

Participants

- Authorizer (**Handler**)
- ConcreteAuthorizer (**ConcreteHandler**).

Collaboration

- A request goes to the first authorizer. If it is allowed to decide, it does. If not (or it does not respond in time), it passes the request to the next one in the chain

What varies

- The number of levels, their order and what limits they may have

Why appropriate

- The chain is assembled per process, so a new level is a new link and nothing else changes.